

AI Engineer Roadmap: Zero to Hired in 6 Months

Who this is for: Students with no prior coding experience who want to become job-ready AI Engineers — not researchers, not ML PhDs. The people building real LLM-powered products that companies actually pay for.

Time commitment: ~15–20 hours/week for 6 months. Less, and you'll stretch this to 9–12 months. More, and you can compress it to 4.

The philosophy:

- Engineering first, then AI on top
 - Build something every single week (even if it's small)
 - Evaluation is the #1 skill that separates AI engineers from "API wrappers"
 - 3 portfolio projects on GitHub > any certificate
-

The 6-Month Overview

Month	Focus	Build by end of month
1	Python, Git & Engineering Basics	A CLI app pushed to GitHub
2	Data, Text & Embeddings Foundations	A semantic search prototype
3	Generative AI & Prompt Engineering	An LLM-powered data extractor
4	RAG, Vector Stores & Frameworks	A "Chat with your docs" app
5	Machine Learning Foundations	A re-ranker that boosts your RAG
6	Agentic Systems, Production & Capstone	A deployed AI product on GitHub

Month 1 — Python, Git & Engineering Basics

Week 1: Python Fundamentals

- Learn variables, data types, conditionals, and loops
- Understand how Python executes code line by line
- Practice reading and debugging error messages
- **Build:** Basic calculator and number guessing game

Week 2: Functions & Data Structures

- Learn functions, arguments, return values, scope
- Work with lists, dictionaries, tuples, and sets
- Start writing cleaner, reusable code
- **Build:** Student marks analyzer and to-do list app

Week 3: OOP & File Handling

- Learn classes, objects, and inheritance
- Read and write files (CSV, JSON, text)
- Handle errors and exceptions cleanly
- **Build:** Mini library management system

Week 4: Git, GitHub & APIs

- Master Git basics: commit, branch, merge, push
 - Call public REST APIs using [requests](#)
 - Push your first project to GitHub
 - **Build:** Weather CLI app using a public API
-

Month 2 — Data, Text & Embeddings Foundations

Week 5: Data Handling with Pandas

- Load CSV and Excel files with Pandas
- Filter, sort, and group data
- Practice exploratory data analysis
- **Build:** Analyze a small real-world dataset

Week 6: Data Cleaning & Preparation

- Handle missing values and duplicates
- Learn basic feature engineering
- Understand why clean data beats fancy models
- **Build:** Clean and prep a messy public dataset

Week 7: Text Processing Basics

- Understand unstructured text data
- Tokenization, stopwords, and normalization
- Intro to regex for cleaning text
- **Build:** Text cleaner and basic tokenizer

Week 8: Embeddings Intuition

- Understand what embeddings are and why they matter

- Use `sentence-transformers` to embed text
 - Compute cosine similarity between texts
 - **Build:** Semantic search over your own notes
-

Month 3 — Generative AI & Prompt Engineering

Week 9: LLM Fundamentals

- Learn how LLMs work at a high level
- Understand tokens, context windows, inference
- Learn the limitations of LLMs
- **Build:** Simple chatbot using an LLM API

Week 10: Prompt Engineering

- Role-based prompting and few-shot examples
- Practice structured outputs (JSON mode)
- Learn prompt iteration techniques
- **Build:** Prompt-based document summarizer

Week 11: Using GenAI APIs

- OpenAI and Anthropic API basics
- Handle responses, errors, and streaming
- Track tokens, rate limits, and costs
- **Build:** Multi-model API wrapper with cost tracking

Week 12: Function Calling & Tool Use

- Define tools and structured outputs
 - Let an LLM call your Python functions
 - Chain multiple tool calls reliably
 - **Build:** LLM-powered data extractor for PDFs
-

Month 4 — RAG, Vector Stores & Frameworks

RAG appears in ~36% of all AI engineering jobs. This is your single biggest leverage month.

Week 13: RAG Fundamentals

- Understand why RAG exists
- Learn RAG architecture components
- Understand chunking strategies

- **Build:** Document chunking pipeline

Week 14: Vector Databases

- How vectors are stored and searched
- Try pgvector, Chroma, or Pinecone
- Perform similarity queries at scale
- **Build:** Vector search system over real docs

Week 15: Building RAG Systems

- Combine retrieval with generation
- Learn context injection techniques
- Add reranking for better answers
- **Build:** Document Q&A bot on your own data

Week 16: AI Frameworks

- Learn LangChain or LlamaIndex basics
 - Compare framework vs from-scratch RAG
 - Add observability with LangSmith or Langfuse
 - **Build:** Production-style RAG with logging
-

Month 5 — Machine Learning Foundations

Not classical ML for its own sake — ML that makes you a better AI engineer.

Week 17: ML Basics & Problem Framing

- Learn what ML is and what it is not
- Supervised vs unsupervised learning
- Frame a real problem as an ML task
- **Build:** ML problem definition document

Week 18: Core ML Models

- Linear and logistic regression intuition
- KNN and decision trees
- Train simple models with scikit-learn
- **Build:** House price or text classification model

Week 19: Model Evaluation

- Accuracy, precision, recall, F1
- Train/test split and cross-validation
- Diagnose overfitting and underfitting
- **Build:** Evaluation report for your classifier

Week 20: ML for AI Engineers

- Embeddings deep dive (revisit + extend)
 - Train a re-ranker to boost RAG accuracy
 - When fine-tuning is and isn't worth it
 - **Build:** ML-enhanced re-ranker plugged into your RAG from Month 4
-

Month 6 — Agentic Systems, Production & Capstone

Week 21: Agentic Workflows

- Learn agent planning and reasoning
- Build with tool calling and orchestration
- Know when agents are unnecessary
- **Build:** Agent that automates a real workflow

Week 22: Evaluation & Optimization

- LLM-as-judge and golden datasets
- Understand cost, latency, and token usage
- Caching and prompt-compression strategies
- **Build:** Evaluation and cost analysis report

Week 23: Production Readiness

- Logging, monitoring, prompt versioning
- Deployment basics (Docker, cloud)
- Responsible AI: safety, PII, prompt injection
- **Build:** Deployed AI app with observability

Week 24: Capstone & Portfolio

- Pick a real domain problem (your domain — fintech, edtech, healthtech, anything)
 - Ship an end-to-end AI product
 - Write a one-pager: problem, evals, tradeoffs
 - **Build:** Portfolio-grade capstone project on GitHub
-

Free Resources Worth Your Time

- **LLM Zoomcamp** (DataTalks.Club) — free, open-source course covering LLMs and RAG
- **AI Engineering Field Guide** by Alexey Grigorev — research from 2,445 real job postings
- **Hamel Husain's blog** — the gold standard on AI evals

- **Eugene Yan's blog** — applied LLM patterns, written by a practitioner
 - **What We Learned from a Year of Building with LLMs** (Applied LLMs, 2024)
 - **Anthropic + OpenAI docs** — read them like a textbook, not a reference
-

The 3 Mistakes That Kill Most Students

1. **Watching tutorials without building.** If you finish a week and don't have a project on GitHub, you didn't do that week.
 2. **Skipping evaluation.** Anyone can call an API. People who can prove their AI system actually works get hired.
 3. **Building generic projects.** "I made a chatbot" gets you 0 interviews. "I built a RAG system that helps lawyers find precedents in 200ms with 92% retrieval accuracy" gets you 5.
-

Your First Week — Start Here

1. Install Python and VS Code today
2. Make a GitHub account
3. Push a "hello world" repo by end of day
4. Start Week 1 tomorrow

The roadmap is just paper until you write your first line of code.

Made this roadmap useful? Save this doc, share it with one friend who's stuck, and follow [\[@aryanwadhavekar\]](#) for the next one.

1. Month 1 — Backend & Async Execution

- **Async Python ([asyncio](#)):** Pelajari [async/await](#) dan penanganan *I/O-bound tasks*. Seluruh pemanggilan LLM API sifatnya *asynchronous*.
- **FastAPI Fundamentals:** Buat REST API modern dengan validasi data otomatis menggunakan Pydantic.
- **Streaming Responses (SSE):** Implementasikan *Server-Sent Events* untuk mengirimkan respons LLM secara *realtime* (efek mengetik seperti di ChatGPT) ke frontend.

2. Month 3 — Structured Outputs & Local Models

- **Structured Output Engine:** Gunakan library **Instructor** atau Pydantic untuk memastikan output JSON dari LLM selalu valid tanpa *parsing error*.
- **Local SLM/LLM Execution:** Jalankan model *open-source* (Llama, Qwen, DeepSeek) di mesin lokal menggunakan **Ollama** atau **vLLM**.
- **Vision & Multimodal Prompting:** Proses dokumen non-teks (PDF bertabel, skema, atau invoice) menggunakan Vision Language Models (VLM).

3. Month 4 — Enterprise RAG & Advanced Retrieval

- **Hybrid Search:** Gabungkan *Keyword/Sparse Search (BM25)* dengan *Semantic/Dense Search* (Vector) untuk menangani pencarian istilah teknis, ID, dan nama spesifik.
- **Reciprocal Rank Fusion (RRF):** Algoritma untuk me-ranking dan menyatukan hasil dari BM25 dan Vector Search.
- **Document Parsing:** Gunakan tool seperti [LlamaParse](#) atau [unstructured](#) untuk mengekstrak tabel dan struktur kompleks dari dokumen PDF sebelum di-*chunking*.

4. Month 6 — Stateful Agents, Security & Performance

- **Graph-Based Agent Frameworks:** Pelajari **LangGraph** (atau konsep *Finite State Machine*) untuk membangun *agentic loop* yang memiliki memori, kontrol alur ketat, dan fitur *Human-in-the-Loop* (HITL).
- **AI Safety & Guardrails:** Implementasikan proteksi *Prompt Injection*, penyaringan PII (Data Pribadi), dan *content moderation* menggunakan **NeMo Guardrails** atau **Llama Guard**.
- **Semantic Caching:** Gunakan **Redis** atau **GPTCache** untuk menyimpan respons dari *prompt* yang mirip guna menekan biaya token dan latensi hingga 0ms.

Ringkasan Pemetaan Penyisipan (Quick Mapping)

Minggu Ke-	Modul Baru yang Disisipkan

Week 4	Tambahkan FastAPI + Async + Streaming Endpoint (Gantikan CLI App biasa).
Week 11	Tambahkan Local LLM Execution (Ollama) sebagai alternatif Cloud API.
Week 12	Gunakan Instructor / Pydantic untuk <i>Structured Outputs & Tool Calling</i> .
Week 14	Masukkan BM25 + Hybrid Search & RRF sebelum masuk ke <i>Reranking</i> .
Week 21	Upgrade Agent biasa ke Stateful Agent (LangGraph) .
Week 23	Masukkan Guardrails (Safety) dan Semantic Caching pada tahap <i>Production</i> .

Need to Complete in FreeCodeCamp

1. Wajib Ditamatkan (Pondasi ML)

Selesaikan modul-modul ini secara berurutan. Ini materi yang bakal lu pakai seumur hidup di Machine Learning:

- **Loops and Sequences:** Pahami manipulasi `List` dan perulangan `for/while` sampai lincah.
- **Dictionaries and Sets: Sangat krusial.** Data mentah, konfigurasi model, dan dataset JSON di ML semuanya berbentuk *Dictionary*.
- **Build a User Configuration Manager:** Kerjain proyek ini buat melatih logika *dictionary* dan *loops*.
- **Error Handling:** Wajib paham `try/except` agar *pipeline* olah data lu gak gampang *crash* pas nemu data busuk.
- **Classes and Objects & Object-Oriented Programming (OOP): SENSASIONAL PENTINGNYA.** Lu gak akan bisa pakai library ML seperti PyTorch kalau gak paham OOP, karena bikin model PyTorch selalu pakai `class MyModel(nn.Module)`.
- **Build a Budget App (atau Polygon Area Calculator):** Pilih salah satu proyek ini buat membuktikan lu udah paham konsep OOP.

2. Boleh Diskip / Bikin Pusing Doang (CS Theory Trap)

Begitu lu selesai sampai bagian **OOP** di atas, **STOP** dan jangan buang waktu di modul-modul berikut:

- **Linear Data Structures & Build a Hash Table (SKIP):** Lu gak perlu bikin *Hash Table* dari nol. Di dunia ML, struktur data tingkat rendah seperti ini sudah diurus sama library bawaan dan Pandas.
- **Recursion (SKIP):** Fungsi memanggil dirinya sendiri. Jarang banget dipakai di ML harian karena di ML kita lebih fokus ke eksekusi tervektorisasi (NumPy).
- **Algorithms & Tower of Hanoi (FIX SKIP):** Teka-teki teoretis ala kompetisi koding/kuliah CS. Tidak ada korelasinya sama skill ngelatih model ML.
- **Graphs and Trees (SKIP dulu):** Baru relevan kalau lu udah masuk ke topik spesifik seperti *Decision Trees* atau *Graph Neural Networks* di tahap *advanced*, tapi bikin dari nol di Python dasar itu buang waktu.
- **Dynamic Programming (FIX SKIP):** Murni materi akademis Informatika yang bikin pusing pemula.
- **Python Certification Exam (OPSIONAL):** Kalau mau ambil silakan, tapi di industri ML, sertifikat koding dasar FCC gak dilirik recruiter—mereka lebih liat proyek ML lu di GitHub.

Garis Finish Lu di Kurikulum Ini

Titik keluar (*exit point*) lu dari kurikulum ini adalah **setelah menyelesaikan proyek OOP (Build a Budget App atau Build a Polygon Area Calculator)**.

Begitu proyek OOP itu beres dan jalan, lu **resmi lulus Python dasar** dan udah siap 100% buat loncat ke library data ML yang sebenarnya: **NumPy, Pandas, dan Matplotlib**.